

Fast N-body simulation on Unity compute shader

Roman Boiko

DH2323 Computer Graphics and Interaction

Stockholm

2026

Introduction

N-body problem is a classical problem from physics. $N \geq 2$ number of bodies are in gravitational interaction with each other and have certain starting coordinates, masses and velocities. The solution that needs to be found is the future positional configuration of the system. This problem describes many real life astronomical systems like planets and moons, star systems, galaxies, galaxy clusters and can even be applied for the whole universe. Thus, solutions for n body systems can provide insight into how these astronomical systems form. For $N=2$ complete analytical solution exists for any starting conditions, while higher N s require specific configurations of coordinates and velocities. Generally higher N systems are solved numerically using computational algorithms.

In this project an algorithm proposed in GPU Gems 3 book is implemented on Unity engine.

Numerical solutions of the N-body problem

In an N-body problem each body interacts with every other one, so default complexity becomes $O(N^2)$. There is a way to reduce it by approximating clusters of distant bodies as single entities (far-field) and calculating accelerations independently only from close bodies (near-field). This project focuses on near-field calculations where $O(N^2)$ persists, and which is called the all-pairs approach.

All-pairs approach cannot be optimized much, because every interaction is calculated either way. A double cycle going through a matrix of every interaction will always be present. Number of calculations (or flops) needed to get acceleration from a particular interaction is constrained by the physical formula. It can be simplified by having less dimensions, but then simulation loses most of its similarity to real world systems. Improvement potential in an all-pairs approach is offered by optimized data storage and memory allocation.

Usually N-body simulations do not have collision physics as simulated bodies like stars in a galaxy are very unlikely to collide. There exist N-body simulations which include collision modelling, but they are much more computationally intensive and are not covered in this project.

Fast N-body algorithm

Algorithm recreated in this project has been developed in 2007 by Nyland et al and published in the GPU Gems 3 book.¹ At the base of the simulation is the acceleration calculation algorithm, which intakes xyz coordinates and masses of two bodies and outputs acceleration for one of them. Collisions are not modelled and bodies can occupy the same space. The equation used as the basis is the gravitational force equation without multiplication by mass, since only raw acceleration is needed:

$$a = G \frac{M \hat{r}}{r^2 r}$$

Where G is the gravitational constant, M mass of the acting body, r the distance between the bodies and $\hat{r}$ is the distance unit vector.

In this form the equation will run into problems if the double cycle gets to same indices, because that would cause zero division by zero. Similar problem occurs when bodies accidentally happen to be on the same coordinates. To avoid it a softening factor is introduced and the equation in slightly rearranged form now looks like this:

$$a = G \frac{M \hat{r}}{(r^2 + \epsilon^2)^{\frac{3}{2}}}$$

Where ϵ is the softening factor.

C# form of the same equation can be seen in Figure 1. Square of distance plus softening factor is calculated first, then inverse root is taken, raised to the power of 3 and multiplied with G and mass.

```
float3 BodyBodyInteraction(float4 bi, float4 bj, float3 ai) // 20Flops total
{
    float3 r = bj.xyz - bi.xyz; // 3Flops
    float distSqr = dot(r, r) + EPS2; // 6Flops
    float invDist = rsqrt(distSqr); // 1Flop
    float invDist3 = invDist * invDist * invDist; // 2Flops
    float s = G * bj.w * invDist3; // 2Flops
    return ai + r * s; // 6Flops
}
```

Figure 1. Single interaction acceleration calculation.

Number of flops assigned to mathematical operations is the same as in the original article. This is done to allow clear performance comparisons between the Unity and CUDA versions.

To solve the N-body problem this equation now needs to be run N^2 times. In theory, N-body acceleration calculation is infinitely parallelizable because interactions are independent, but in practice this requires too much memory bandwidth as all threads would independently access body data.

Article, on which this project is based proposes a solution to this problem. Core of the approach is dividing matrix of interactions into computational tiles. Tile is a submatrix of the full interaction matrix. If the full matrix has size $N \times N$ then there are $N/P \times P$ tiles. Each tile is calculated sequentially. Inside the tile columns are calculated sequentially as well, while rows are parallelized and each row corresponds to a thread. This means thread count is also P . Each thread updates the acceleration of a row body according to column bodies. A row of tiles forms what is called a thread block.

Tile approach offers multiple memory improvements. Since all threads in tile share a single column they can use same shared data for that one particular body. With the row body information for each thread the total amount of memory to calculate a tile is $2P$. When moving to the next tile in the tile row half of that data can be reused, as row bodies do not change for N/p tiles. Splitting the $N \times N$ matrix into thread blocks horizontally limits the amount of shared memory needed and allows to delegate different thread blocks to multiple microprocessors.

A memory synchronization occurs before the start of tile calculation and another one after. First synchronization is needed to make sure all relevant data is loaded before the calculations, while the second ensures no thread will overwrite the shared memory until all have finished their rows.

Architecture of the N-body problem simulation in Unity

To create an animation in Unity the most straightforward approach is to use GameObjects, which have a controller script attached that updates their position. However, it is extremely inefficient for a very large number of bodies like in a N-body simulation. Instead, only one GameObject is created and then an associated mesh is rendered N times, using a custom shader.

Whole system consists of 3 parts: controller, shader and compute shader. Controller is attached to the empty GameObject and acts as the core script. Controller initiated body data is dispatched to the compute shader. Compute shader calculates accelerations and returns them to the controller. Controller then makes another dispatch to compute shader for integration step. Then after receiving updated positions, controller sends them to the shader, which renders the bodies on the screen. Whole setup can be seen in Figure 2.

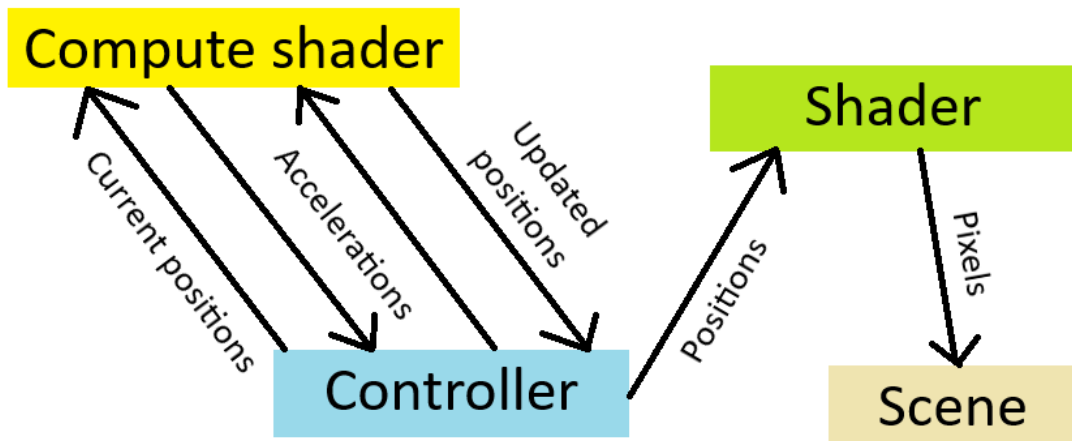


Figure 2. Schematic of Unity N-body simulation. Accelerations are sent back to controller and then again to compute shader for integration.

Implementation

N-body simulation implementation was created using Unity and Microsoft VS Code. First, a project as created in Unity with relevant GameObjects and then supporting scripts were written in VS Code in C# language. Aside from using C#, shader also has its main body written in High-level shading language (HLSL). During the writing of the code generative AI, specifically Claude Haiku 4.5 integrated into VS Code and MS Copilot inside MS Edge, was heavily used, both for questions and code writing. Shader consists almost exclusively from generated code, while other components have roughly 50-50 split between human and machine input.

Controller

Controller script consists of two parts: start and update. Start method is called once at the beginning of the simulation. 3 Vector4 arrays are created for positions/masses, velocities and accelerations. Using Vector4 struct enables to store mass together with coordinates and to use coalesced access because of 16 bit size.¹ Arrays are then populated with body data. Each body gets assigned with random position, mass and initial velocity. After initialization is done the data gets loaded into ComputeBuffers, which are ready to be sent to the compute shader.

Update method is called each frame. It dispatches the acceleration kernel and then the integration kernel after which shader is called and bodies are drawn with updated positions.

Compute shader

Compute shader does the main workload. It consists of two kernels, one for acceleration calculations and one for position and velocity integration. Kernels are called sequentially by the controller script. Compute shader uses ComputeBuffer class variables to store exchanged data. Accelerations are calculated through previously described tile method with a double for loop. Integration is done through a simple leapfrog integrator using the same number of parallel threads as the acceleration kernel.

Shader

Shader is the final part of the program that creates the image in the Unity scene. Shader uses instanced rendering, meaning there is only one draw call, but shader draws multiple meshes based on data provided. It takes in the positions from the controller and then creates and applies a translation matrix for every instance of a body. Shader also applies the specified mesh and color. As it is a simple unlit shader, no

actual shading is done, and all bodies have the same uniform color. Rendering all bodies through this shader is efficient but makes applying different colors and shapes to different bodies more difficult.

Performance measurement

To make objective comparisons of different model parameters with regards to performance, a Gflops/sec counter was created. Knowing that one interaction calculation consists of 20 flops and the total amount of interactions is N^2 , calculated every frame, total amount of flops can be calculated as:

$$Gflops/s = \frac{20N^2 * FPS}{10^9}$$

Where FPS is frames per second. Since instantaneous FPS is highly unstable, averaging method was added, which calculated FPS as total amount of frames in the current run divided by total time elapsed.

This method is not absolutely accurate, as time for the frame generation (frametime) is also affected by all other calculations the computer makes, so real performance is higher. Still, it can be assumed that acceleration calculations are the primary load, and other processes are less significant, so the number is close to reality.

Gflop counter was implemented as a separate controller script and displayed on a text box in Unity scene, which could be seen through game camera during the simulation run.

Hardware and Software

Computer used is an MSI GF75 Thin 10UEK with a Nvidia Geforce RTX 3060 Mobile GPU and Intel Core i7 CPU. This graphics card should be theoretically capable of 10.94 TFLOPS performance.² Simulation is run in Unity 6000.04.0f1 on Windows 11 Home operating system.

Optimization

Before trying to achieve actual simulation results a series of optimization tests was done, mimicking the GPU Gems article. Optimal loop size and tile size were determined. Since performance tests do not depend on a specific body arrangement bodies were initialized as 2 spherical clusters with uniform distribution and circular velocities in xz plane, crudely depicting galaxies (Figure 3).

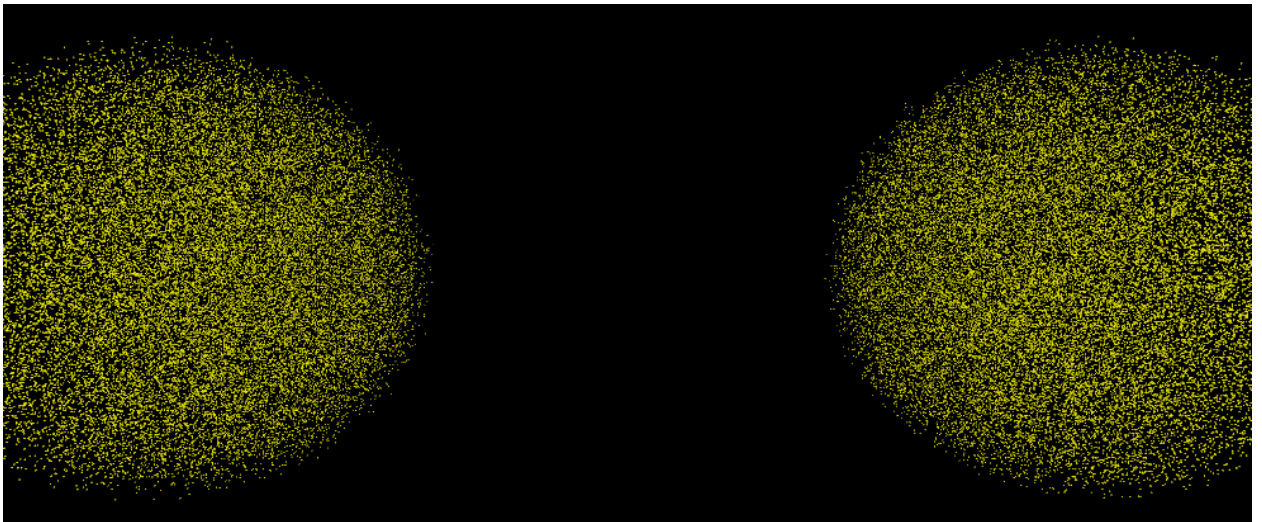


Figure 3. Two body clusters.

Loop unrolling

Nyland et al were able to achieve a slight increase in performance by unrolling the loop by a factor of 4. Loop unroll was introduced into compute shader script by slightly altering the loop (Figure 4). To reproduce the result, simulations for 1,2,4 and 8 unroll count were made. Performance did marginally

increase with unroll count 2 but started to decrease from 4 and onwards (Figure 5). Difference in results can be explained by different loop overheads of the environments in use.

```
for (uint i = 0; i < tileSize; i++)
{
    acc = BodyBodyInteraction(myPosMass, shPos[i], acc);
}

for (uint i = 0; i < tileSize; i += 2)
{
    acc = BodyBodyInteraction(myPosMass, shPos[i], acc);
    acc = BodyBodyInteraction(myPosMass, shPos[i + 1], acc);
}
```

Figure 4. Default (top) and 2 times unrolled (bottom) loop.

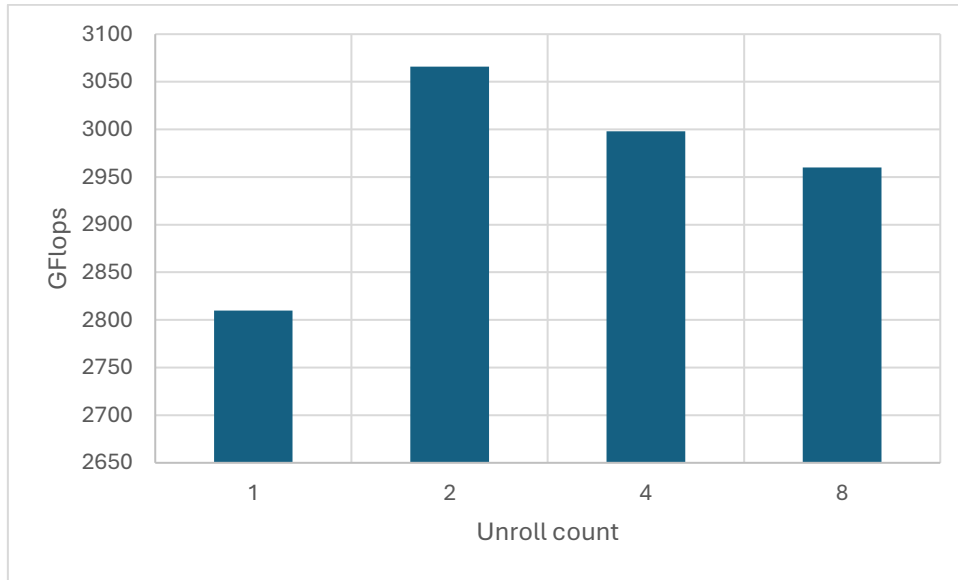


Figure 5. Performance vs unroll count.

Tile size

Tile size can either be too small or too large. Smaller size means less parallelism, while bigger size requires more shared memory. Different tile sizes from 16 to 512 were tested with $N = 65536$. It was found that a tile size of 256 was the most optimal one, though not differing too much from other values. However, tile size of 16 had a significantly worse performance (Figure 6).

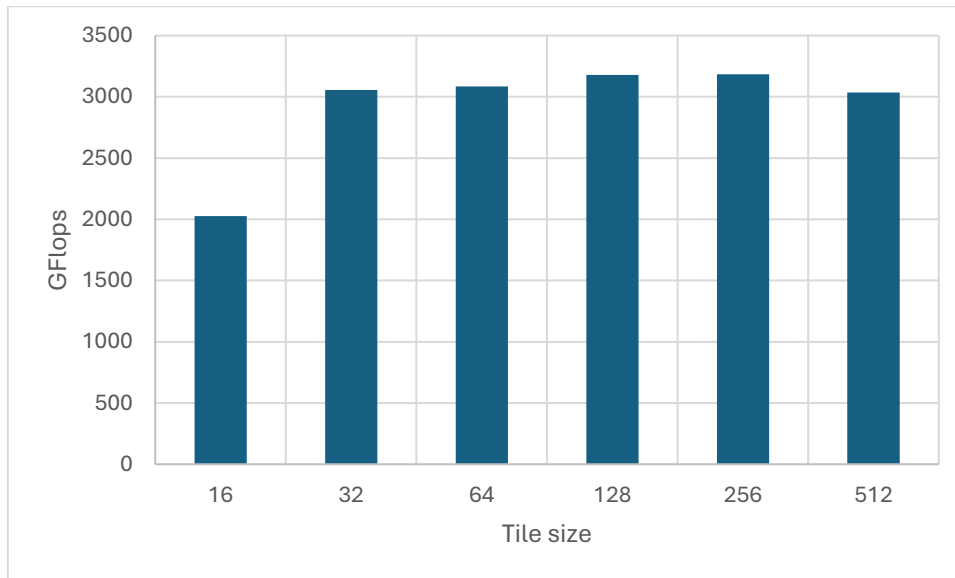


Figure 6. Performance vs tile size.

Performance results

Different body counts were used to see how performance is affected and what is the highest Gflop number that can be reached. Tile size was chosen to be 256 and unroll count 2 based on previous optimization tests. To ensure higher validity, Unity was the only application running at the time of simulations. N tested started from 512 and was increased by a factor of two every time, reaching 524 288. At lower N there is a fast increase in performance, but the growth slows down to a halt as N approaches the higher values (Figure 7).

5.5 Tflops were achieved at maximum body count, which is nearly 28 times higher than the result from the Nyland et al 2007 article. However, authors of that article were able to achieve theoretical maximum of their graphics card using this algorithm, but the current result is different. Maximum performance of the graphics card used should be around 11 Tflops, while only 5.5 is reached. There are multiple reasons as to why that may be the case. Either the algorithm implementation is not as efficient or there are external factors affecting the framerate. It is hard to say where the problem lies, but since Windows Task Manager demonstrates nearly 100% GPU utilization (0% while idle), it is possible the method of calculating flops is just inaccurate and framerate is slowed down because of other processes. However, at low N an FPS of over 200 is seen, which means framerate of other processes can be estimated at around 5 ms. For N=524288 framerate grows to around 1 s, which would not be affected by only a few milliseconds. There are other processes scaling with N, like integration and rendering, but they should be negligible compared to acceleration calculations.

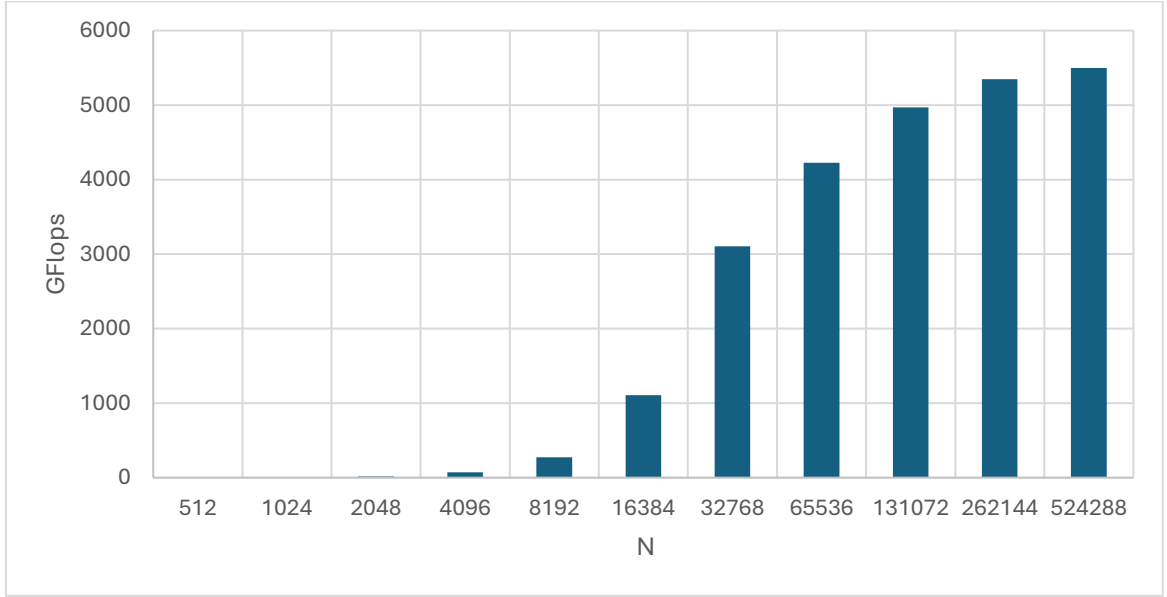


Figure 7. Performance vs number of bodies N .

Simulating collision of two galaxies

One the most popular applications of N-body algorithms are cosmological simulations and galaxy passages in particular. Despite being many decades old such simulations remain relevant up to this day.^{3,4} A scenario of two galaxies passing was chosen to illustrate the feasibility of the current project's algorithm because it is relatively easy to implement and spinning spiral galaxies can offer aesthetically pleasing visuals.

Code expansion

Previous initialization code had bodies spawn in two designated spherical volumes uniformly and hardly resembled actual galaxies, even when assigning circular velocities. To create a proper galaxy model an article by Gerhard et al from 1981 was chosen as an example.⁵

Galaxy model consists of 3 parts: core, disk and halo. Core is a single body in the center of the galaxy, representing gravitational center. Disk is a 2D structure of circularly rotating bodies. Halo is a lump of bodies spread around the center with isotropic velocities. While initializing the core is straightforward, disk and halo required substantial expansions to code. Core is a single body, while disk and halo get equal number of bodies depending on total N , with disk having one less to make the total number even. Every body was given the same mass, while core was given a mass of 10% of the disk and halo mass combined.

Both disk and halo have non-uniform distributions and so two new methods for sampling radii were implemented, based on inverse cumulative distribution function (inverse CDF) sampling, for the most part using generative AI. Inverse CDF sampling works by generating a random value from uniform distribution and then, assuming it is the value of CDF, inversely calculating to which non-uniform x this value corresponds. Disk uses a decaying exponential surface density distribution limited by disk radius and adjusted with parameter α , while halo has inverse square volume density distribution depending on halo radius. Both galaxies are created simultaneously.

For a proper simulation all bodies are assigned with initial velocities. Core is initially at rest. Disk bodies are assigned with circular speed that counteracts the inward radial acceleration:

$$v = \sqrt{a_{rad}r}$$

Where a_{rad} is radial acceleration towards core and r is distance to the core.

Halo bodies are given isotropic velocities equal to square root of half of local gravitational potential that approximately satisfy the virial theorem:

$$v = \sqrt{0.5U}$$

Where U is local gravitational potential.

To calculate initial velocities starting accelerations and potentials were needed. For this an additional one time dispatch of acceleration kernel was added into the initialization code. Additionally, a new potential kernel was created based on the acceleration kernel with slight changes. Both kernels are dispatched after the initial positions assignment and before the velocities initialization. As the acceleration and potential kernels both take into account all bodies present, bodies of one galaxy can be affected by bodies of the other. To minimize this interaction, galaxies are initially assigned far away coordinates and only then are the velocities calculated. At this step an interesting bug was discovered that assigned NaN value to one or more bodies (maximum observed 3 out of 4024). This NaN would generate NaN position values for all other bodies when the first acceleration calculation would be made and then the rendering would stop working. To remedy this a simple „if NaN“ check was made that would replace all NaNs with zeros.

After galaxies have been assigned both positions and velocities, translation to the required starting positions happens. This is done by removing the initial offset and applying user specified distance between two galaxies in three dimensions. Afterwards galaxies are rotated around x and z axes with given angle values. As the last step of initialization, galaxies as a whole are given starting velocities antisymmetrically in x direction to avoid a head-on collision.

Body-wise coloring was added to better visualize different galaxies and their constituent parts. An additional buffer with colors was created to be passed to the shader. Colors then were made to be assigned together with positions. To achieve a high contrast between bodies, first galaxy was assigned orange disk and aquamarine halo, while the second was assigned lime disk and blue halo. Both cores are red, but they were hardly visible behind a large number of bodies.

All the relevant parameters, like galaxy positions, disk and halo radii, individual body mass were inputted through Serializefields.

New galaxy model resembled an actual galaxy much more accurately. With a higher N, galaxies started forming galactical arms and with enough time central bar structures would also appear (Figure 8). Both are real observed features of galaxies.^{4,6}

A big drawback of this model is that it does not consider dark matter, understandably as the article is from 1981. However, the point of using this model is to demonstrate the feasibility of the fast N-body algorithm and not provide an accurate description of real astronomical processes, so that can be overlooked.

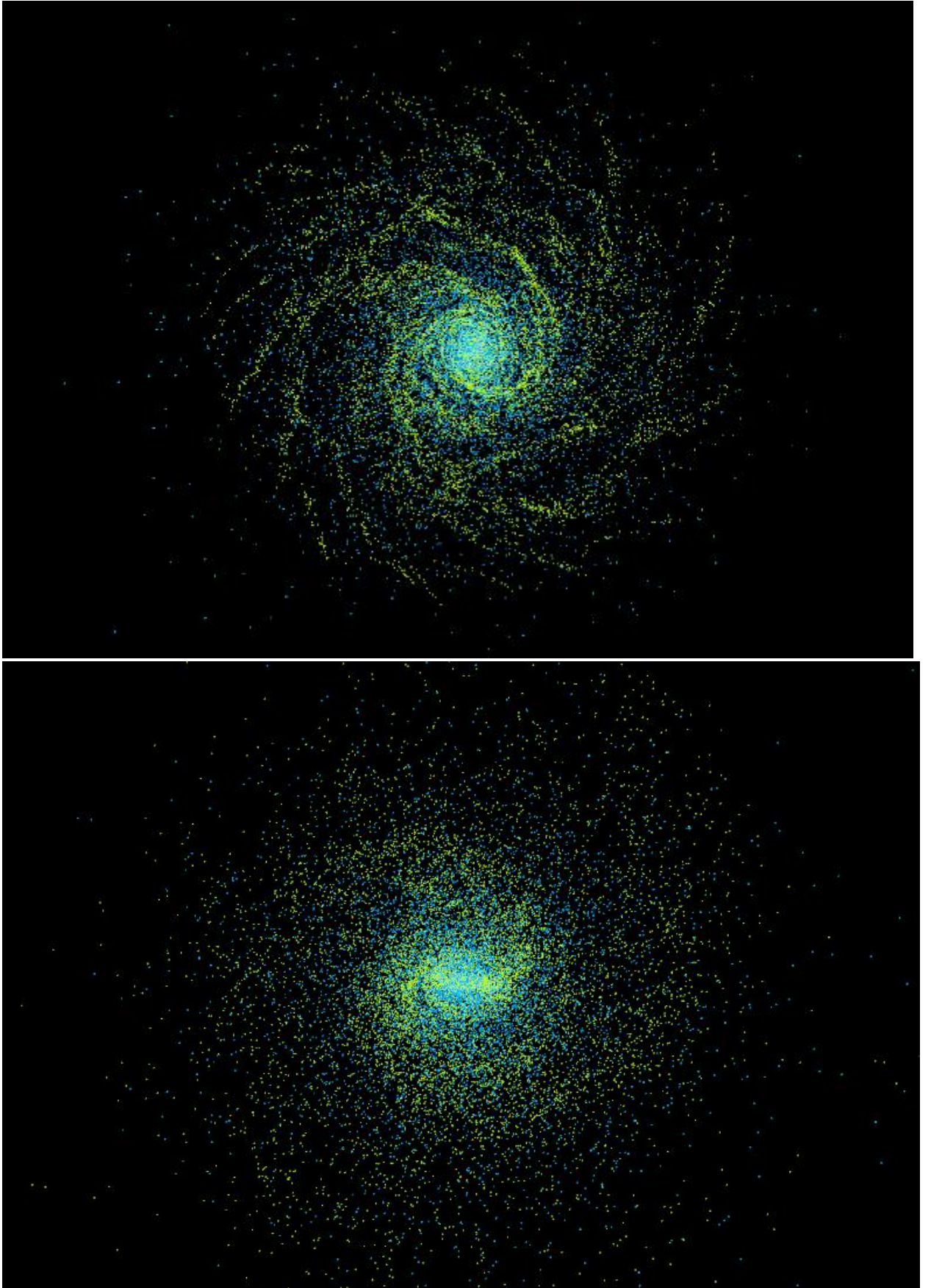


Figure 8. Galaxy from 20120 bodies. Halo bodies are colored blue, disk bodies lime. Lime galaxy arms are visible (top). Central bar structure forming after some time (bottom).

Passage tests

To evaluate the simulation algorithm multiple passage tests were made, mimicking the parabolic passage scenarios from Gerhard et al. A parabolic passage is a type of passage when trajectories are parabolic and galaxies do not end up locked in a system. Tests that were done in this project were not strictly speaking parabolic, as galaxies did end up merging after a very long time, but nonetheless are quite close to the intended event. Initial parameters were recalculated from the article to fit the new simulation environment units.

Two different scenarios were studied. In the first scenario (A) galaxies move clockwise and have their spins oriented at arbitrary angles (specified in the Gerhard 1981). In the second scenario (B), spins are antiparallel and one of the spins is parallel with the orbital movement of the whole system. The intended outcome was for galaxies to interact largely symmetrically in the first scenario, while minor asymmetry was to appear in the second scenario.

Each scenario was tested 2 times with different total body count N , 1024 and 10240 respectively, to see if any differences arise. As with higher body count the galaxies need to be bigger to retain roughly the same density, disk and halo radii and initial distance were adjusted accordingly (Table 1).

Test nr	1	2	3	4
Scenario	A	A	B	B
N	1024	10240	1024	10240
Disk radius	20	200	20	200
Halo radius	25	250	25	250
Single body mass	1	1	1	1
Core mass	51.2	512	51.2	512
G	1	1	1	1
Softening factor	10	10	10	10
alpha	0.01	0.01	0.01	0.01
Initial distance	100	800	100	800
Angle 1, x	-26	-26	0	0
Angle 1, z	-14	-14	0	0
Angle 2, x	6	6	180	180
Angle 2, z	-57	-57	0	0
Initial velocity	0.5	0.5	0.5	0.5
Time step	0.02	0.02	0.02	0.02

Table 1. Test parameters. Angles are around the specified axis and in degrees, other parameters are arbitrary simulation units.

Test results were indeed as expected. Both $N=1024$ and $N=10240$ simulations showed symmetrical galaxy sizes after the passage at arbitrary orientation (Figures 9,10), meanwhile in the case with antiparallel spins slight asymmetry was noticeable (Figure 10,11). Varying N numbers also caused differences. At high N galactical arms formed and later faded while central bar structure appeared, at the same time low N simulations had none of those features. Additionally, in the B scenario bar of the galaxy which was antiparallel was more protruded, while the galaxy which was parallel to orbital movement formed a rounder bar.

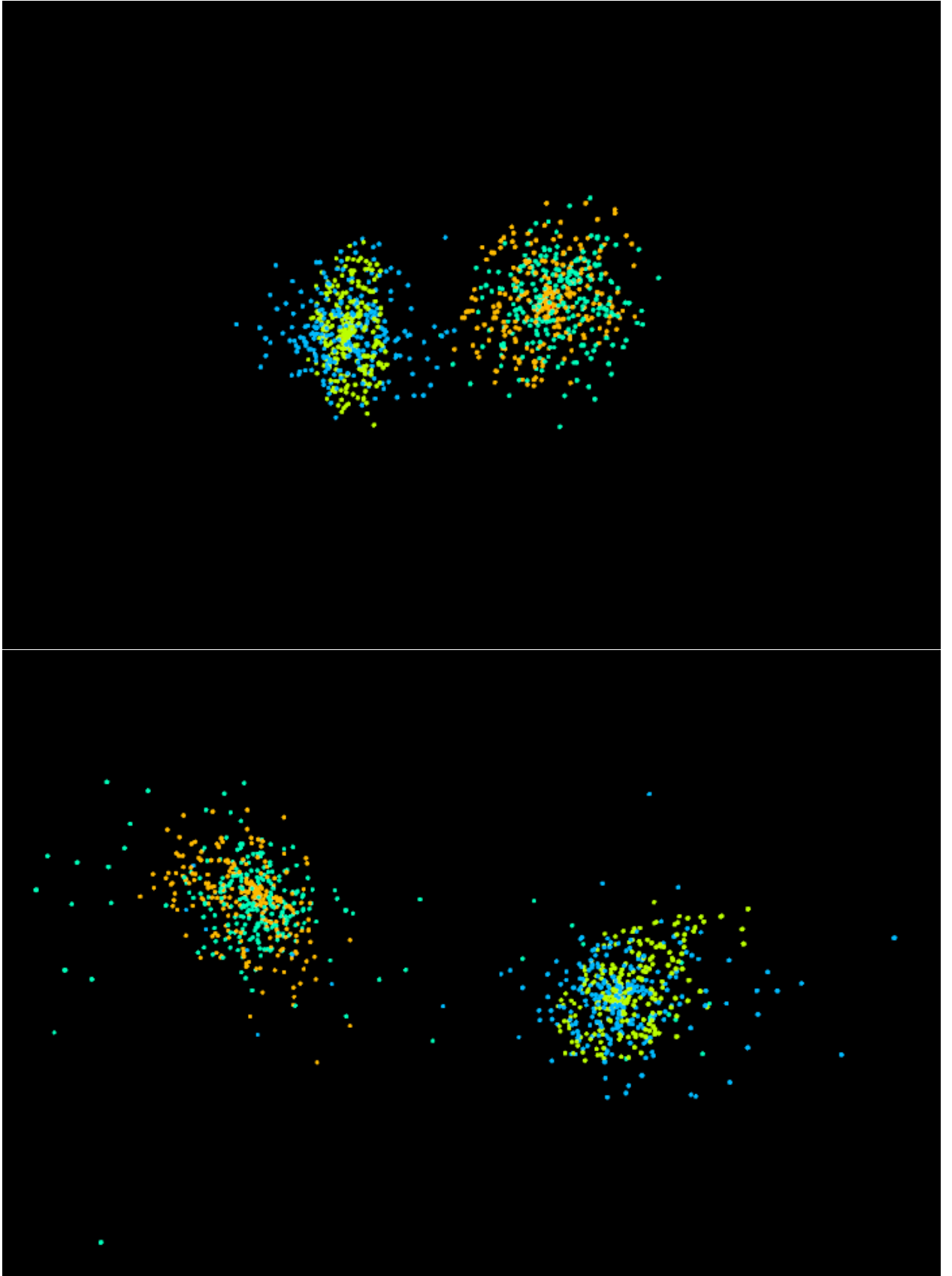


Figure 9. Test nr 1. $N=1024$, arbitrary angles. Galaxies at the closest passage point (top) and at the largest distance after the first passage (bottom). Galaxies remain largely the same, some escaping bodies can be seen.

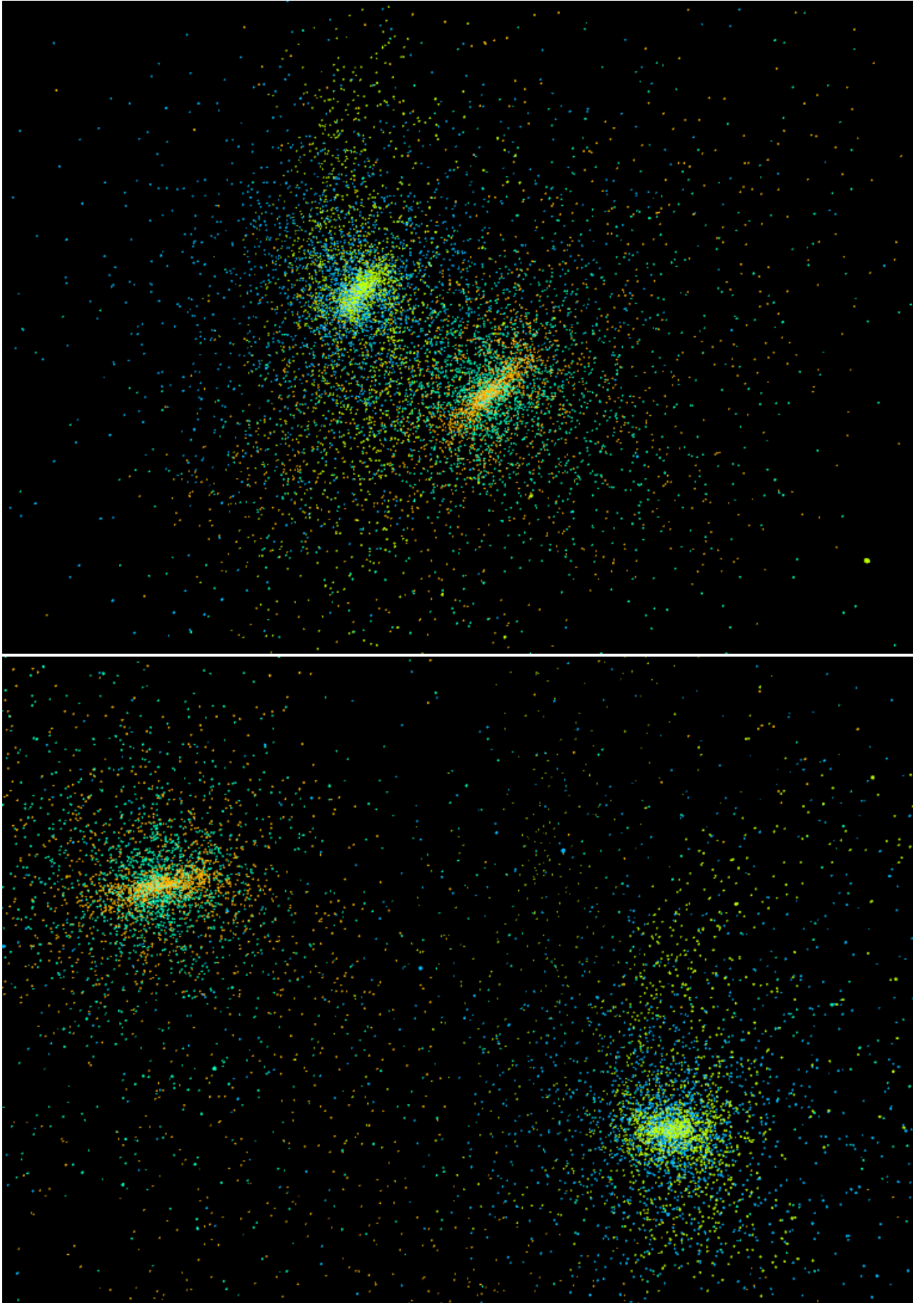


Figure 10. Test nr 2. $N=10240$, arbitrary angles. Galaxies at the closest passage point (top) and at the largest distance after the first passage (bottom). Bar structures are visible, apparent different length is actually caused by different galaxy angles.

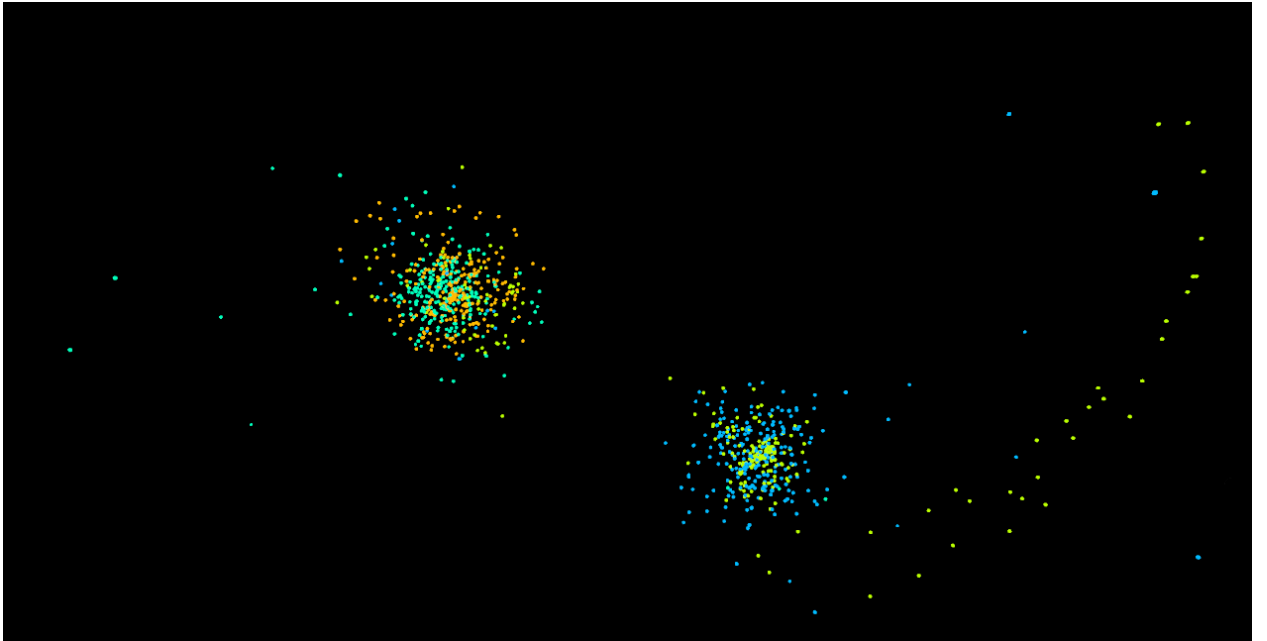
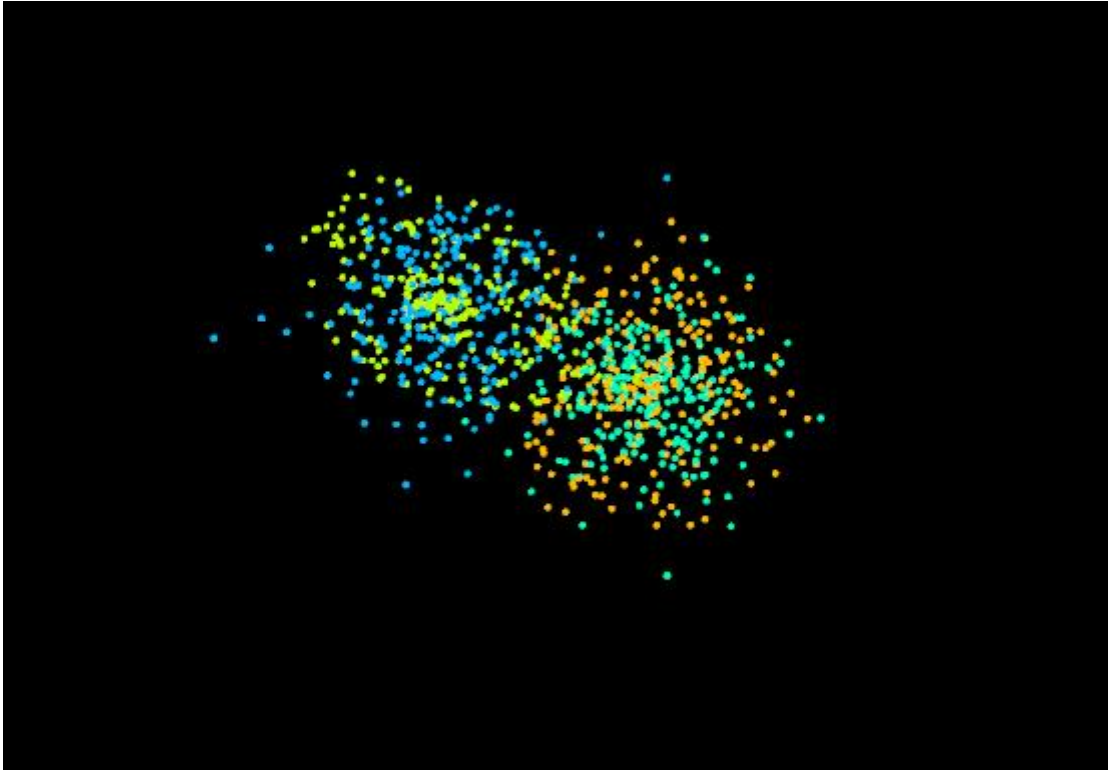


Figure 11. Test nr 3. $N=1024$, antiparallel angles. Galaxies at the closest passage point (top) and at the largest distance after the first passage (bottom). Galaxy 1 (orange/aquamarine) captures some bodies from the other galaxy. Galaxy 2 (lime/blue) clearly gets smaller and ejects more disk bodies.

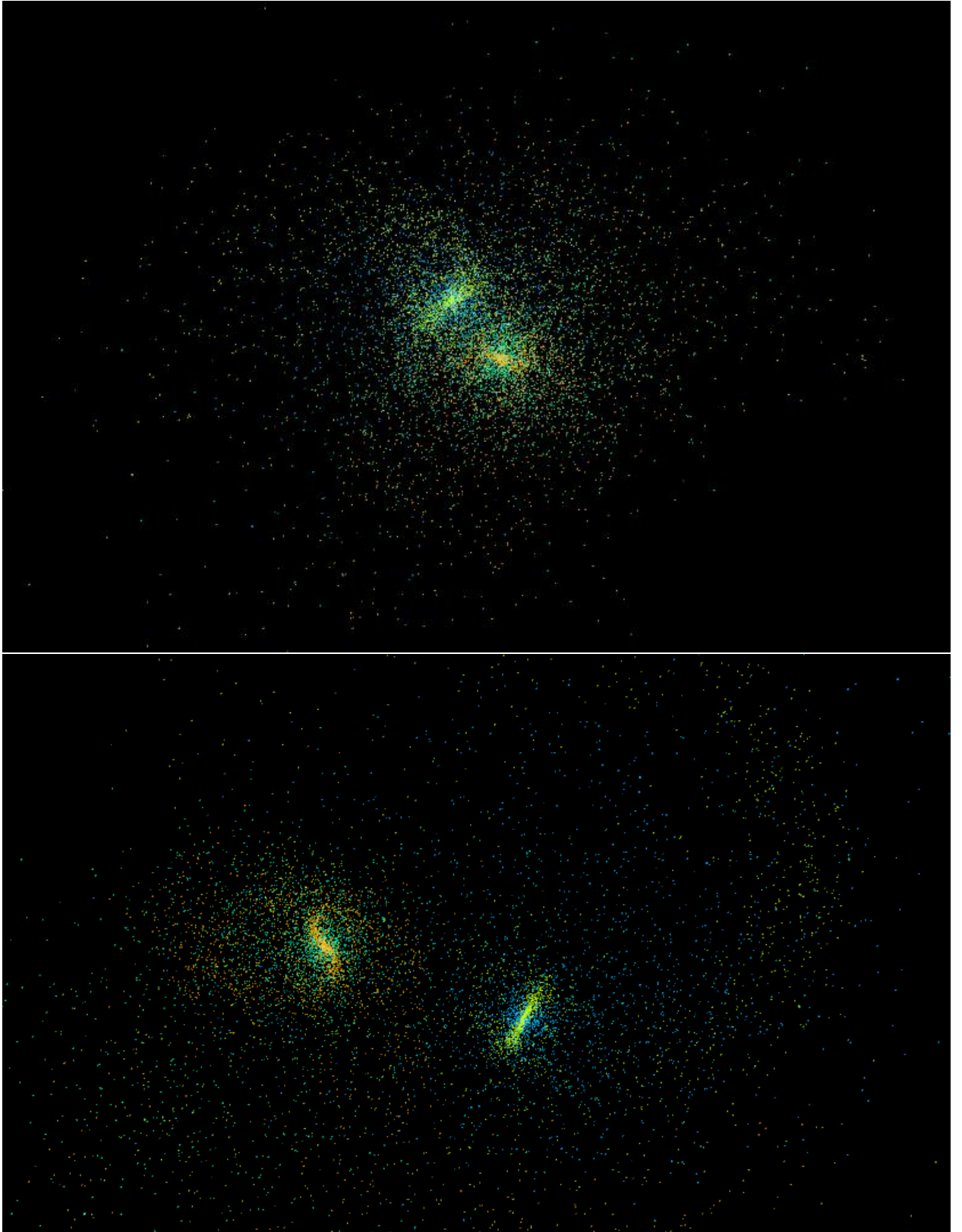


Figure 12. Test nr 4. $N=10240$, antiparallel angles. Galaxies at the closest passage point (top) and at the largest distance after the first passage (bottom). Galaxy 1 (orange/aquamarine) has a denser cloud around the core, while galaxy 2 (lime/blue) appears less bright.

Natural galaxy coloring

To showcase the graphical capabilities of the program, natural coloring for galaxies was implemented with some minor additions and changes to the code. To showcase new coloring 2 galaxies were merged into one by removing initial offset and inputting zero as well. 2 arrays with different colors for disk and

halo were created and then those colors were assigned to relevant galaxy parts. Colors then were adjusted based on distance to the core so center would appear brighter and yellower. It was found that halo parts would obstruct the disk and so number of halo particles was reduced. To do this dynamically a new user provided variable “halodisk_ratio” was added, that specifies what is the ratio between number of disk and halo bodies. This variable was chosen to be 0.1. N was increased to 99840 to make it look smoother while keeping the radii 200 and 250 for disk and halo respectively. Result can be seen on Figure 13. Difference with previous coloring can be seen on Figure 14, where test nr 4 is replicated.

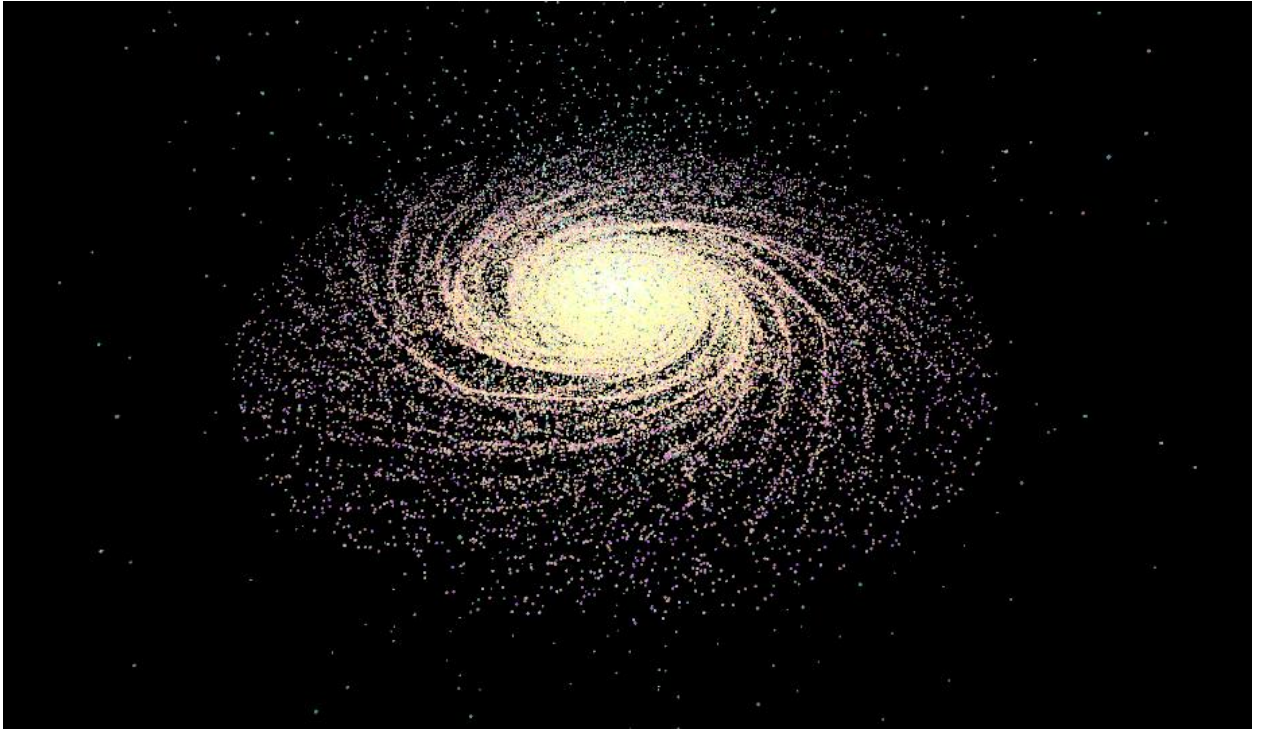


Figure 13. Naturally colored galaxy from 99840 bodies.



Figure 14. Test nr 4 remade with natural galaxy coloring.

Conclusion

The project of recreating the fast N-body algorithm for CUDA in Unity was for the most part successful. Maximum theoretical performance of the GPU was unfortunately not achieved, but the algorithm still performed quite well, getting as high as 120 FPS at 32k bodies. Maximum number of bodies simulated was 524k but the frametime increases so much that it becomes unwatchable. Still, such high N numbers can be used for research purposes, with visualization being done later, if at all. Loop unroll and tile size optimizations yielded slightly different results than in the article this work is based upon with smaller unroll count being better while the same tile size being the most optimal.

Many technical challenges were encountered, especially in the beginning, and the project took more time than planned but it turned out well enough in the end. Expanding and tuning the program to get realistic galaxy models went much smoother, as it was technically simpler and a lot of project-relevant knowledge was acquired by that point.

Simulating galaxy passages demonstrated the program's potential to do useful tests for research. In all tests the program managed to simulate results of passage, showing differences in interactions depending on starting galaxy orientation, as expected by previous research. However, galaxy models should be expanded for doing actual astrophysical studies, as they are still quite simplistic and lack dark matter.

Future improvements to this code can be made. Aside from expanding the galaxy model, optimizing and compartmentalizing galaxy initialization algorithm can make the code more flexible to different simulation scenarios. Currently two galaxies are created at once in single code section. A better way would be to have a galaxy initialization method that is called multiple times with translation being a separate callable method as well. This would allow to change the number of galaxies and make adding other astronomical structures as separate methods easier.

Sources

1. Nylons, L., Harris, M. & Prins, J. Fast n-body simulation with cuda. *GPU gems* **3**, 62–66 (2007).
2. NVIDIA GeForce RTX 3060 Mobile. *TechPowerup* <https://www.techpowerup.com/gpu-specs/geforce-rtx-3060-mobile.c3757>.
3. Vergara, M. C. *et al.* Rapid formation of a very massive star ($> 50000 M_{\odot}$), and subsequently, of an IMBH, from runaway collisions: Direct N-body and Monte Carlo simulations of dense star clusters. *Astron. Astrophys.* **704**, A321 (2025).
4. Zhou, Y. F., Li, Z. & Li, H. N-body simulations of galaxy bars generated by satellite collisions: effects of the impact geometry. *Astrophys. J.* **988**, 205 (2025).
5. Gerhard, O. E. N-body simulations of disc–halo galaxies: isolated systems, tidal interactions and merging. *Mon. Not. R. Astron. Soc.* **197**, 179–208 (1981).
6. Smith, B. J., Giroux, M. L. & Struck, C. The effect of environment on galaxy spiral arms, bars, concentration, and quenching. *Astron. J.* **164**, 146 (2022).